GREYNOC

C2 & INTRUSION RESPONSE

Windows + Linux Field Manual

Tracking command-and-control, investigating remote code execution, containing live intrusions, and eradicating malicious persistence

OPERATOR DOCTRINE: Preserve evidence. Reduce attacker options. Verify every conclusion. Do not confuse geography, a strange process name, or one antivirus alert with proof of compromise. Build a chain of evidence.

GreyNOC Defensive Operations Series

Edition 1.1 | Operator Reference | Accuracy review: August 2026

Read This First

This manual is written for an operator who believes a Windows or Linux system may be under active attack. It assumes you may be looking at confusing symptoms: unexpected outbound traffic, an account logging in from somewhere unfamiliar, a browser claiming you are in another country, a suspicious script in Startup, a Defender alert, a strange service, or a process that keeps returning after you kill it.

The goal is not to teach exploitation. The RCE material is defensive: how to recognize evidence that remote code execution occurred, how to determine what executed, how to contain it, and how to remove persistence and recover safely. Where commands can alter a machine, the manual explicitly labels them as containment or eradication steps.

LEGAL AND OPERATIONAL SCOPE: Use these procedures only on systems you own or are authorized to administer. If the host belongs to an employer, customer, school, or other organization, follow its incident-response policy and escalation path.

The evidence ladder

Signal	What it can tell you	What it cannot prove
IP geolocation	What a database believes about a public address	Who is controlling a machine or whether traffic is malicious
Open connection	A process is communicating with a remote endpoint	That the endpoint is C2
Process path + signature	Whether a binary looks like expected installed software	That the process has not been injected into or abused
Persistence artifact	How code may survive reboot/logon	Whether it is malicious without context
Execution evidence	What actually ran and when	Initial access by itself
Correlated timeline	Multiple independent events that support one story	Absolute certainty; incident response is probabilistic

The core rule: do not destroy your evidence

- If the host is actively being attacked and the stakes are high, prefer network isolation over immediately powering it off. RAM may contain injected code, decrypted configuration, active sockets, and credentials that disappear at shutdown.
- Do not randomly delete files, clear logs, uninstall software, or run “cleaner” utilities before you know what happened.
- Document the system time, your actions, suspicious PIDs, file paths, hashes, remote addresses, accounts, and timestamps.
- If you must contain quickly, block or isolate first; eradicate second.

Part I - Live Attack Triage

Chapter 1 - The First Five Minutes

When an intrusion may be live, the first job is to decide whether you are dealing with a real security event, ordinary software behavior, or an environmental problem such as VPN routing, DNS, proxy configuration, or a stale geolocation database. You do not need to solve the entire incident in five minutes. You need to stop the situation from getting worse while preserving enough evidence to explain it later.

1. Establish a timestamp and operator log

Write down the local time, timezone, hostname, logged-in user, and what caused suspicion. Record commands as you run them. A simple text log is sufficient if no ticketing system exists.

```
Windows
Get-Date
hostname
whoami

Linux
date --iso-8601=seconds
hostnamectl
id
```

2. Decide whether immediate isolation is justified

Isolate when you see strong evidence of ongoing unauthorized control: ransomware activity, confirmed credential theft, a remote shell you did not establish, an unknown process repeatedly beaconing to a known-malicious destination, active data staging, or an attacker changing accounts or security controls.

> **CONTAINMENT CHOICE:** If you need forensic evidence, disconnect the network cable, disable Wi-Fi, or place the host in a containment VLAN rather than shutting it down. If human safety or destructive malware is involved, stopping the system may take priority over evidence preservation.

3. Capture volatile network state

```
Windows
Get-NetTCPConnection | Sort-Object State,OwningProcess
Get-NetUDPEndpoint | Sort-Object OwningProcess
arp -a
ipconfig /all
route print

Linux
ss -tupna
ip addr show
ip route show
ip neigh show
```

Look for repeated outbound connections from unexpected processes, unusual high-frequency reconnects, unknown listeners, or connections that do not fit the machine's role. A single HTTPS connection to a cloud provider is not enough.

4. Capture process state

```
Windows
Get-CimInstance Win32_Process | Select ProcessId,ParentProcessId,Name,ExecutablePath,CommandLine |
Sort ProcessId

Linux
ps -eo pid,ppid,user,lstart,cmd --sort=pid
ps auxww
```

Parent-child relationships are often more revealing than a process name. For example, an Office application spawning PowerShell, a web server spawning a shell, or a service daemon launching curl followed by a newly downloaded binary deserves immediate attention.

5. Check the obvious security controls

```
Windows
Get-MpComputerStatus | Format-List
AntivirusEnabled,RealTimeProtectionEnabled,BehaviorMonitorEnabled,AntivirusSignatureLastUpdated
Get-MpThreatDetection | Sort InitialDetectionTime -Descending | Select -First 20 | Format-List *

Linux
systemctl --failed
journalctl -p warning..alert --since "2 hours ago"
```

Do not treat the absence of antivirus alerts as proof the machine is clean. The value is correlation: did a detection occur at the same time as the suspicious process or connection?

Chapter 2 - The First Fifteen Minutes

After the volatile snapshot, move from “what is happening now?” to “what mechanism is making it happen?” Build a shortlist of suspicious processes, accounts, services, tasks, scripts, and remote endpoints. For each candidate, collect enough information to either explain it or escalate it.

Question	Why it matters
What process owns the connection?	C2 is executed by code. The process is usually more useful than the IP alone.
What is the executable path?	Malware often runs from user-writable or deceptive locations.
Who signed the binary?	A valid signature is useful context, though signed software can still be abused.
What is the parent process?	RCE and LOLBin abuse often produce unusual parent-child chains.
How did it persist?	Persistence explains why an attacker returns after a reboot or process kill.
What account context is involved?	User, SYSTEM/root, service, and container contexts change the blast radius.
What changed recently?	File, service, task, package, account, and log changes anchor the timeline.

Chapter 3 - C2 Is a Behavior, Not a Country

Command-and-control is the communication path an attacker uses to operate compromised code. It may use dedicated infrastructure, a cloud platform, a CDN, an anonymous VPN, a compromised server, a messaging API, DNS, HTTPS, WebSockets, SSH, or even software already permitted by the organization. The physical or registered country of an IP address is weak attribution evidence.

VPNs make this especially obvious. If a website says the system is in India, Germany, Brazil, or another country and reconnecting the VPN changes the result, the simplest explanation is usually the VPN exit IP or inconsistent geolocation. To investigate security, compare the public IP before and after the symptom and correlate it with the local process/socket table.

Baseline your public egress address

```
Windows or Linux
curl.exe -s https://api.ipify.org     # Windows curl.exe
curl -s https://api.ipify.org         # Linux
```

DO NOT OVER-ATTRIBUTE: "This IP geolocates to India" is not equivalent to "an attacker in India controls this machine." Treat geography as enrichment, not proof.

Chapter 4 - A Practical C2 Hunting Model

A useful C2 hunt asks four questions in order: (1) which process communicated, (2) what initiated that process, (3) what mechanism keeps it available to the attacker, and (4) what else happened around the same time. This model works on both Windows and Linux and prevents tunnel vision around IP reputation.

C2 indicators that become stronger when combined

- Regular beaconing intervals from an application that normally should not make network connections.
- A new or unsigned binary in a user-writable directory with persistent outbound traffic.
- A service, scheduled task, cron job, systemd unit, shell profile, or Run key that launches the same binary.
- A process tree showing a document reader, browser child process, web service, or scripting engine producing an unexpected shell or downloader.
- DNS requests for algorithmically generated or newly observed domains followed by HTTPS connections.
- Connections that continue when the legitimate application they supposedly belong to is closed.
- EDR/Defender/audit events showing injection, credential access, tampering, or suspicious script execution at the same time.

Weak indicators that need context

- Foreign IP address
- Port 443
- A process named svchost.exe
- A high-numbered local port
- A file in AppData or /tmp
- One antivirus PUA detection
- A process using PowerShell or curl

Part II - Windows 11 / Windows Server

Chapter 5 - Windows Network Investigation

Windows gives you enough built-in visibility to perform a strong first-pass investigation without installing third-party tools. Start by mapping connections to PIDs, then map PIDs to process paths and command lines.

Established TCP connections

```
Get-NetTCPConnection -State Established |
  Select-Object LocalAddress,LocalPort,RemoteAddress,RemotePort,OwningProcess |
  Sort-Object OwningProcess
```

Listeners

```
Get-NetTCPConnection -State Listen |
  Select-Object LocalAddress,LocalPort,OwningProcess |
  Sort-Object LocalPort
```

Map PIDs to full process data

```
$ids = 1234,5678
Get-CimInstance Win32_Process |
  Where-Object { $_.ProcessId -in $ids } |
  Select-Object ProcessId,ParentProcessId,Name,ExecutablePath,CommandLine |
  Format-List
```

Interpretation

A connection becomes interesting when ownership does not fit the application. A browser connecting to a CDN is ordinary. A newly created executable in a Temp directory repeatedly connecting to the same external address every 30 seconds is not. A Windows service host connecting to Microsoft endpoints may be normal; the service name behind that svchost instance tells you more.

```
Get-CimInstance Win32_Service |
  Where-Object ProcessId -eq 1234 |
  Select Name,DisplayName,State,StartMode,PathName
```

Chapter 6 - Windows Process and Binary Validation

Authenticode signature

```
Get-AuthenticodeSignature 'C:\Path\program.exe' | Format-List
Status,StatusMessage,SignerCertificate,Path
```

A valid signature means the file has a cryptographically valid signature from the listed publisher. It does not prove the software is harmless. Attackers can abuse legitimate signed tools, and signed binaries can be vulnerable or compromised.

Hash the binary

```
Get-FileHash 'C:\Path\program.exe' -Algorithm SHA256
```

Record the SHA-256 before quarantine or removal. Hashes let you correlate the same file across endpoints and with threat-intelligence systems without moving the executable around.

Inspect file metadata

```
Get-Item 'C:\Path\program.exe' | Select FullName,Length,CreationTime,LastWriteTime,VersionInfo |
Format-List
```

Suspicious process-tree patterns

Parent	Child	Why to investigate
winword.exe / excel.exe	powershell.exe, cmd.exe, wscript.exe	May indicate malicious document or macro execution
w3wp.exe / httpd service	cmd.exe, powershell.exe	May indicate web application RCE or web shell activity
browser.exe	unexpected native binary from Downloads/Temp	Potential drive-by or malicious download execution
services.exe	unknown binary in AppData/Temp	Unusual service persistence
taskeng.exe / svchost.exe	script engine with encoded or hidden arguments	Possible scheduled persistence or defense evasion

Chapter 7 - Windows Persistence

If a suspicious process returns after you terminate it or reboot the machine, find the persistence mechanism before repeatedly killing the process. Persistence is often the shortest path to understanding the intrusion.

Startup commands

```
Get-CimInstance Win32_StartupCommand |
  Select-Object Name,Command,Location,User |
  Format-List
```

Run and RunOnce keys

```
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run' -ErrorAction SilentlyContinue
Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run' -ErrorAction SilentlyContinue
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\RunOnce' -ErrorAction
SilentlyContinue
Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\RunOnce' -ErrorAction
SilentlyContinue
```

Scheduled tasks, including what they execute

```
Get-ScheduledTask | ForEach-Object {
  foreach ($a in $_.Actions) {
    [PSCustomObject]@{
      Task = "$($_.TaskPath)$($_.TaskName)"
      Execute = $a.Execute
      Args = $a.Arguments
    }
  }
} | Format-List
```

Services

```
Get-CimInstance Win32_Service |
  Select Name,DisplayName,State,StartMode,StartName,PathName |
  Sort StartMode,Name
```

WMI permanent event subscriptions

```
Get-CimInstance -Namespace root/subscription -ClassName __EventFilter
Get-CimInstance -Namespace root/subscription -ClassName CommandLineEventConsumer
Get-CimInstance -Namespace root/subscription -ClassName ActiveScriptEventConsumer
Get-CimInstance -Namespace root/subscription -ClassName __FilterToConsumerBinding
```

FIELD NOTE: WMI permanent event subscriptions are uncommon on many workstations. They are not automatically malicious, but unknown command-line or script consumers deserve careful investigation.

Chapter 8 - Windows Event Logs and Defender

Defender detections

```
Get-MpThreatDetection |
  Select InitialDetectionTime,ThreatID,ThreatStatusID,Resources,ActionSuccess |
  Sort InitialDetectionTime -Descending |
  Format-List

Get-MpThreat | Format-List
ThreatID,ThreatName,SeverityID,CategoryID,DidThreatExecute,IsActive,Resources
```

PowerShell operational log

```
Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' -MaxEvents 200 |
  Select TimeCreated,Id,LevelDisplayName,Message | Format-List
```

Process creation if Security 4688 auditing is enabled

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4688; StartTime=(Get-Date).AddHours(-24)} |
  Select TimeCreated,Id,Message | Format-List
```

Service installation event 7045

```
Get-WinEvent -FilterHashtable @{LogName='System'; Id=7045; StartTime=(Get-Date).AddDays(-7)} |
  Select TimeCreated,Message | Format-List
```

Logon events

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624; StartTime=(Get-Date).AddHours(-24)} |
  Select TimeCreated,Message | Format-List
```

Remote logons are not inherently malicious. Investigate source addresses, logon type, account, workstation name, and whether the login fits expected administration. Correlate successful logons with process creation and network activity.

Chapter 9 - Windows RCE Investigation

Remote code execution means an attacker caused code to execute on the machine through a remote-facing service, application vulnerability, malicious content, management interface, stolen credentials, or another remote mechanism. Your job is to prove the execution chain, not recreate the exploit.

What to look for

- A network-facing service spawning a shell or scripting engine.
- A service account creating files in web roots, Temp, ProgramData, or user profile directories.
- New services or tasks created shortly after unusual inbound connections.
- PowerShell script-block or operational events that coincide with the suspected compromise.

- Web server access/error logs showing unusual requests immediately before a new process or file appears.
- Unexpected local administrator creation or group membership changes.
- Credential use from a remote host followed by process execution.

Find recent executable/script writes

```
$since=(Get-Date).AddDays(-2)
Get-ChildItem C:\Users,C:\ProgramData,C:\Windows\Temp -Recurse -Force -ErrorAction SilentlyContinue |
  Where-Object { $_.LastWriteTime -ge $since -and $_.Extension -match
'\.(exe|dll|ps1|bat|cmd|vbs|js|hta)$' } |
  Select FullName,Length,CreationTime,LastWriteTime |
  Sort LastWriteTime -Descending
```

PERFORMANCE WARNING: Recursive searches can be expensive. Narrow the path and time window when investigating production systems.

Check local administrators

```
Get-LocalGroupMember -Group 'Administrators'
```

Check recently created local users

```
Get-LocalUser | Select Name,Enabled,LastLogon,PasswordLastSet,Description
```

Chapter 10 - Windows Containment

Containment should reduce attacker control without erasing the evidence needed to understand the incident. If you have centralized EDR isolation, use it. On a standalone workstation, physical or network isolation may be faster and safer than attempting dozens of firewall rules while an attacker is active.

Disable a suspicious scheduled task

```
Disable-ScheduledTask -TaskName 'SuspiciousTask' -TaskPath '\'
```

Stop and disable a confirmed malicious service

```
Stop-Service -Name 'BadService' -Force
Set-Service -Name 'BadService' -StartupType Disabled
```

Temporarily block a confirmed malicious remote IP

```
New-NetFirewallRule -DisplayName 'IR block 203.0.113.10' -Direction Outbound -Action Block -
RemoteAddress 203.0.113.10
```

CONTAINMENT WARNING: Do not block an address merely because it is foreign or unfamiliar. Shared cloud/CDN addresses may affect legitimate software and are weak indicators without process-level evidence.

Chapter 11 - Windows Eradication and Recovery

Eradication starts only after you have identified the malicious mechanism with reasonable confidence. Remove persistence, quarantine the malicious payload, reset affected credentials, patch the entry point, and verify that the activity does not return.

Move to an evidence holding directory rather than immediately delete

```
New-Item -ItemType Directory -Path 'C:\IR-Holding' -Force
Move-Item 'C:\Path\suspect.exe' 'C:\IR-Holding\suspect.exe'
Get-FileHash 'C:\IR-Holding\suspect.exe' -Algorithm SHA256
```

A manually created holding directory is not antivirus/EDR quarantine and does not by itself prevent execution. Moving malware is only appropriate after execution has been stopped and persistence disabled. In higher-assurance environments, use EDR/Defender quarantine or acquire a forensic image instead of manually handling the file.

Remove a confirmed malicious scheduled task

```
Unregister-ScheduledTask -TaskName 'SuspiciousTask' -TaskPath '\' -Confirm:$false
```

Remove a confirmed malicious service

```
sc.exe delete BadService
```

Remove a malicious Run value

```
Remove-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run' -Name 'BadValue'
```

Run Defender scans after containment

```
Update-MpSignature
Start-MpScan -ScanType QuickScan
# For a deeper scan when operationally appropriate:
Start-MpScan -ScanType FullScan
```

When to reimage instead of clean

Reimage when you cannot confidently bound the compromise, when privileged credential theft is likely, when rootkit/boot-level tampering is suspected, when critical security controls were disabled or altered in ways you cannot reconstruct, or when the host is high-value and assurance matters more than preserving the installed state. Cleaning a machine is not the same as proving it trustworthy.

Part III - Linux Workstations and Servers

Chapter 12 - Linux Network Investigation

The same evidence ladder applies on Linux: socket to PID, PID to executable and parent, process to persistence, then timeline. The command names differ but the logic is identical.

Sockets and listeners

```
sudo ss -tupna
sudo lsof -nP -i
sudo ss -lntup
```

Inspect the owning process

```
ps -fp 1234
tr '\0' ' ' < /proc/1234/cmdline; echo
readlink -f /proc/1234/exe
ls -l /proc/1234/cwd
cat /proc/1234/status
```

Parent process chain

```
pstree -aps 1234
ps -o pid,ppid,user,lstart,cmd -p 1234
```

Chapter 13 - Linux Binary and File Validation

Hash a suspicious file

```
sha256sum /path/to/suspect
```

Identify file type

```
file /path/to/suspect
stat /path/to/suspect
```

Determine package ownership

```
# Debian/Ubuntu
dpkg -S /usr/bin/example
dpkg -V package-name

# RHEL/Fedora
rpm -qf /usr/bin/example
rpm -V package-name
```

Package verification can reveal files that differ from the vendor package, but configuration files may legitimately change. Treat verification output as a lead, not an automatic verdict.

Chapter 14 - Linux Persistence

systemd services and timers

```
systemctl list-unit-files --type=service
systemctl list-timers --all
systemctl --failed

# Inspect a unit
systemctl cat suspicious.service
systemctl show suspicious.service
```

Cron

```
crontab -l
sudo crontab -l
sudo ls -la /etc/cron.d /etc/cron.daily /etc/cron.hourly /etc/cron.weekly /etc/cron.monthly
sudo cat /etc/crontab
```

Shell profiles and autostart

```
grep -RniE 'curl|wget|bash -c|python|perl|nc|socat|/tmp|/dev/shm' \
  ~/.bashrc ~/.profile ~/.bash_profile /etc/profile /etc/profile.d 2>/dev/null

find ~/.config/autostart -maxdepth 1 -type f -print 2>/dev/null
```

SSH authorized keys

```
find /home /root -path '*/.ssh/authorized_keys' -type f -print -exec sed -n '1,200p' {} \;
2>/dev/null
```

DO NOT DELETE UNFAMILIAR KEYS BLINDLY: Servers may have automation, backup, deployment, and break-glass keys. Compare fingerprints and owners against authorized administration records before removal.

Chapter 15 - Linux Logs and Authentication

Recent system journal

```
sudo journalctl --since '24 hours ago'
sudo journalctl -p warning..alert --since '24 hours ago'
```

SSH authentication

```
# Ubuntu/Debian often use /var/log/auth.log
sudo grep -iE 'Accepted|Failed|Invalid user|session opened|sudo' /var/log/auth.log | tail -n 300

# RHEL-like systems often use /var/log/secure
sudo grep -iE 'Accepted|Failed|Invalid user|session opened|sudo' /var/log/secure | tail -n 300
```

Login history

```
last -ai | head -n 100
lastb -ai | head -n 100   # requires permissions and failed-login database
```

sudo activity

```
sudo journalctl _COMM=sudo --since '24 hours ago'
```

Chapter 16 - Linux RCE Investigation

On Linux servers, RCE often becomes visible as a service process spawning something it should not: a web server spawning /bin/sh, a Java application spawning curl or wget, a database service writing an executable into /tmp, or a container process creating an unexpected host process. The goal is to find the causal chain from remote request or login to executed code.

Look for service-to-shell ancestry

```
ps -eo pid,ppid,user,lstart,cmd --forest
pstree -ap
```

Recent files in common staging locations

```
sudo find /tmp /var/tmp /dev/shm -xdev -type f -mtime -2 -printf '%TY-%Tm-%Td %TH:%TM:%TS %u %m %s
%p\n' 2>/dev/null | sort -r
```

Recent executable files

```
sudo find /tmp /var/tmp /dev/shm /home -xdev -type f -perm /111 -mtime -2 -ls 2>/dev/null
```

Web service logs

```
# Common examples; paths vary by distribution/application
sudo tail -n 500 /var/log/nginx/access.log
sudo tail -n 500 /var/log/nginx/error.log
sudo tail -n 500 /var/log/apache2/access.log
sudo tail -n 500 /var/log/apache2/error.log
```

Correlate request timestamps with process start times and file creation times. A suspicious request is much stronger evidence when it is immediately followed by a service child process and a newly written payload.

Chapter 17 - Linux Rootkits, Injection, and Hidden Activity

Do not assume every discrepancy means a rootkit. Start with simpler explanations such as deleted-but-running files, mount namespaces, containers, package updates, and permissions. Then escalate if multiple independent views disagree.

Deleted executable still running

```
sudo lsof +L1
# or targeted
sudo ls -l /proc/*/exe 2>/dev/null | grep '(deleted)'
```

Kernel modules

```
lsmod
modinfo module_name
```

Loaded shared libraries for a process

```
cat /proc/1234/maps | sed -n '1,200p'
```

Compare process views

```
ps auxww
sudo find /proc -maxdepth 1 -regex '/proc/[0-9]+' -printf '%f\n' | sort -n | tail
```

Chapter 18 - Linux Containment

Stop and disable a confirmed malicious systemd service

```
sudo systemctl stop suspicious.service
sudo systemctl disable suspicious.service
```

Block a confirmed malicious outbound address with nftables

```
sudo nft add table inet ir
sudo nft 'add chain inet ir output { type filter hook output priority 0; policy accept; }'
sudo nft add rule inet ir output ip daddr 203.0.113.10 drop
```

FIREWALL CAUTION: Do not improvise firewall policy on remote production systems without an out-of-band recovery path. You can lock yourself out just as effectively as you can lock out an attacker.

Lock a compromised local account

```
sudo usermod -L username
```

Locking a local password does not automatically invalidate SSH keys, active sessions, API tokens, or application credentials. Treat identity containment as a separate workstream.

Chapter 19 - Linux Eradication and Recovery

Disable persistence before moving the payload

```
sudo systemctl stop suspicious.service
sudo systemctl disable suspicious.service
sudo mkdir -p /root/ir-quarantine
sudo sha256sum /path/to/suspect
sudo mv /path/to/suspect /root/ir-quarantine/
```

Remove a malicious cron entry

```
crontab -e
# For root, when authorized:
sudo crontab -e
```

Remove an unauthorized SSH key

First record the key, its fingerprint, file owner, permissions, and timestamps. Then edit the relevant authorized_keys file and remove only the confirmed unauthorized line.

```
ssh-keygen -lf /home/user/.ssh/authorized_keys
stat /home/user/.ssh/authorized_keys
```

Patch and restart the exploited service

Do not bring a previously vulnerable service back online until the underlying vulnerability or exposure is addressed. Recovery may mean applying a vendor patch, replacing the package, disabling an unnecessary module, changing configuration, restricting network access, rotating credentials, and reviewing dependent systems.

Part IV - Cross-Platform Deep Dive

Chapter 20 - Building a Timeline

Timelines turn a pile of artifacts into an incident story. Use a consistent timezone, preferably UTC for multi-system incidents. Record events with source, timestamp, host, user/process, action, and confidence. The sequence matters: remote request -> process start -> file write -> persistence -> outbound connection -> credential use is very different from a benign updater that creates a process and contacts its vendor CDN.

Time	Source	Observed event	Interpretation
14:02:11	Web access log	Unusual POST to vulnerable endpoint	Possible initial trigger
14:02:12	Process log	Web worker spawns shell	Strong RCE evidence
14:02:13	Filesystem	New binary written to Temp	Payload staging
14:02:15	Task/service	Persistence created	Attacker durability
14:02:18	Network	New process connects externally	Potential C2
14:07:40	Security log	New admin account	Privilege/persistence expansion

Chapter 21 - DNS, Proxies, VPNs, and False Positives

Network environment issues can imitate compromise symptoms. A VPN can change public IP and geolocation. A secure web gateway can make many hosts appear to originate from one address. DNS filtering can redirect domains. Browser extensions and corporate proxies can alter what websites see. Always document the egress path before escalating a geographical anomaly into an intrusion claim.

Windows proxy state

```
netsh winhttp show proxy
Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings' | Select
ProxyEnable,ProxyServer,AutoConfigURL
```

Windows DNS

```
Get-DnsClientServerAddress
Resolve-DnsName example.com
```

Linux proxy and DNS

```
env | grep -i proxy
resolvectl status 2>/dev/null || cat /etc/resolv.conf
getent hosts example.com
```

Chapter 22 - Credential and Identity Response

If the attacker reached administrator, SYSTEM, root, a domain account, cloud token, browser session, or SSH key, assume credential material may have been exposed. Removing malware without rotating compromised secrets can leave the attacker a clean way back in.

- Identify which accounts were logged in or used while the compromise was active.
- Revoke active sessions and tokens through the relevant identity provider.
- Rotate passwords, API keys, service-account secrets, and SSH keys according to impact.
- Review new MFA methods, recovery options, forwarding rules, OAuth grants, and application passwords.
- Check neighboring systems for reuse of the same credentials or keys.

Chapter 23 - Evidence Preservation

For a low-risk home workstation, screenshots, command output, hashes, and exported logs may be enough. For legal, regulatory, enterprise, or high-impact incidents, preserve evidence according to formal forensic procedure. Do not alter original media when a forensic image is required.

Windows export example

```
wevtutil epl System C:\IR\System.evtx
wevtutil epl Security C:\IR\Security.evtx
wevtutil epl 'Microsoft-Windows-PowerShell/Operational' C:\IR\PowerShell-Operational.evtx
```

Linux journal export example

```
sudo journalctl --since '2026-08-19 00:00:00' --until '2026-08-20 00:00:00' -o short-iso > journal-
2026-08-19.txt
```

Chapter 24 - Deciding Clean, Contain, Rebuild, or Escalate

Situation	Recommended direction
Single PUA file, never executed, quarantined	Remove source copies, scan, verify; rebuild usually unnecessary
Known malware executed as standard user, bounded persistence	Contain, investigate scope, eradicate, rotate exposed credentials, validate
Confirmed admin/root compromise	Strongly consider rebuild; rotate privileged credentials and inspect peers

Situation	Recommended direction
Unknown kernel/boot tampering	Rebuild from trusted media; preserve forensic evidence
Ransomware/destructive behavior	Immediate isolation, activate incident plan, preserve evidence, recover from trusted backups
Cannot explain persistence or repeated reinfection	Escalate and rebuild rather than repeatedly “cleaning” symptoms

Chapter 25 - Validation After Eradication

The incident is not over when the suspicious file is gone. Validation proves that persistence, credentials, entry point, and secondary artifacts were addressed.

1. Reboot at least once when operationally safe, then repeat connection, process, startup/service/task, and account checks.
2. Confirm the suspicious destination no longer receives traffic from the implicated process or replacement process.
3. Confirm security controls are enabled and current.
4. Run an appropriate antivirus/EDR scan.
5. Verify patched versions and hardened configuration for the original entry point.
6. Review identity logs for post-cleanup access using old credentials.
7. Monitor for recurrence over a meaningful window based on the threat and system role.

Chapter 26 - The GreyNOC Live-Incident Decision Tree

STEP 1: Do you have destructive activity, confirmed remote shell, credential theft, or a strongly malicious process? If yes: isolate network and preserve evidence. If no: continue triage.

STEP 2: Can you map the suspicious network activity to a process and executable? If no: improve visibility. If yes: validate the binary, parent process, command line, and persistence.

STEP 3: Is there execution evidence or only a suspicious file? If only a file: determine whether it executed before escalating scope. If executed: build the timeline.

STEP 4: Was privilege obtained? If admin/root or sensitive credentials were exposed: expand scope and favor rebuild/credential rotation.

STEP 5: Do you know the entry point and persistence? If not: do not declare the system clean. Continue investigation or rebuild.

STEP 6: After eradication, does the activity return after reboot and monitoring? If yes: assume missed persistence, missed credential path, or reinfection source.

Part V - Advanced Investigation and Casework

Chapter 27 - Capturing Network Evidence

A socket table tells you who is connected at one moment. Packet capture can show connection frequency, DNS lookups, TLS handshakes, retransmissions, and protocol behavior over time. Capture only when authorized and when it will not expose unrelated sensitive traffic unnecessarily. Modern HTTPS usually prevents you from seeing application content, but metadata alone can still be valuable.

Windows built-in packet capture with pktmon

```
pktmon start --capture --pkt-size 0 --file-name C:\IR\capture.etl
# Reproduce or observe the suspicious behavior for a short, controlled period.
pktmon stop
pktmon etl2pcap C:\IR\capture.etl --out C:\IR\capture.pcapng
```

Linux packet capture with tcpdump

```
sudo tcpdump -i any -nn -s 0 -w /root/ir-capture.pcap
# Stop with Ctrl+C after the observation window.
```

Do not run unlimited captures on busy systems. Use a defined observation period and sufficient free disk space. If a destination is already known, a narrow capture filter can reduce noise, but preserve an unfiltered short capture when you need context around DNS and related connections.

Chapter 28 - Improving Windows Visibility with Sysmon

Sysmon can provide high-value process creation, network connection, file creation, image-load, DNS, and other telemetry when it has been installed and configured before the incident. Do not install it mid-incident on a high-stakes forensic target without considering how the installation will change evidence. If it is already present, use its Operational log as a correlated source.

```
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 300 |
  Select TimeCreated,Id,Message | Format-List
```

Common Sysmon event	Typical use
1 - Process Create	Parent/child, command line, hashes when configured
3 - Network Connection	Process-to-destination correlation when enabled
7 - Image Load	DLL/module loading investigations when enabled
11 - File Create	Payload and staging-file timeline
13 - Registry Value Set	Registry persistence/configuration changes
22 - DNS Query	Process-associated DNS investigation

Configuration matters. An event type may be disabled or heavily filtered. Absence of a Sysmon event is not proof an action did not occur.

Chapter 29 - Improving Linux Visibility with auditd

Linux audit telemetry can connect file execution, account actions, privilege use, and configuration changes to users and processes. As with Sysmon, it is most valuable when configured before the incident.

```
sudo auditctl -s
sudo ausearch -ts today
sudo ausearch -m USER_LOGIN,USER_AUTH,USER_START -ts today
sudo aureport -au
```

Audit rules vary widely. Do not assume the host records every exec or file write. Determine what rules are active before drawing conclusions from missing events.

Chapter 30 - Remote Access Software: Legitimate Tool or Attacker Channel?

Attackers frequently abuse legitimate remote administration products because the software is signed, encrypted, and expected to communicate through firewalls. The presence of a remote-access tool is not automatically malicious. The investigative question is whether it was authorized, who installed it, when it first appeared, which account controls it, and whether its sessions match legitimate administration.

Windows discovery ideas

```
$uninstall = 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*',`
             'HKLM:\Software\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*',`
             'HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*'
Get-ItemProperty $uninstall -ErrorAction SilentlyContinue |
  Where-Object DisplayName -match 'AnyDesk|TeamViewer|ScreenConnect|ConnectWise|Splashtop|RustDesk' |
  Select-Object DisplayName,DisplayVersion,Publisher,InstallLocation

Get-CimInstance Win32_Service |
  Where-Object { $_.Name -match 'AnyDesk|TeamViewer|ScreenConnect|ConnectWise|Splashtop|RustDesk' -or
$_.PathName -match 'AnyDesk|TeamViewer|ScreenConnect|ConnectWise|Splashtop|RustDesk' } |
  Select-Object Name,State,StartMode,PathName
```

Linux discovery ideas

```
ps auxww | grep -Ei 'anydesk|teamviewer|screenconnect|connectwise|splashtop|rustdesk'
systemctl list-unit-files | grep -Ei
'anydesk|teamviewer|screenconnect|connectwise|splashtop|rustdesk'
```

DO NOT REMOVE BY PRODUCT NAME ALONE: A help-desk tool, MSP agent, remote-work product, or accessibility tool may be legitimate and business-critical. Validate authorization and session history first.

Chapter 31 - Web Application and Service RCE Casework

When a network-facing application is suspected, build the case around the service. Identify its worker process, service account, listening port, web/access logs, application logs, child processes, recent writes, and configuration changes. Then align them on a timeline.

Windows: identify listener and owning service

```
Get-NetTCPConnection -State Listen | Sort LocalPort
Get-CimInstance Win32_Service | Where-Object ProcessId -eq 1234 | Format-List *
```

Linux: listener, service, and journal

```
sudo ss -lntup
systemctl status nginx
sudo journalctl -u nginx --since '2 hours ago'
```

A compelling RCE chain usually has several links: unusual inbound request or authenticated action, service child process, script/shell execution, payload write or command side effect, then persistence or outbound control. If you cannot establish the chain, label the conclusion with the uncertainty rather than filling gaps with assumptions.

Chapter 32 - Memory and Live-Process Clues

Memory analysis is a specialized forensic discipline, but even without a full memory-image workflow you can capture useful live-process clues. Record process command line, loaded modules/libraries, handles or open files, current directory, network sockets, and parent process before terminating a suspicious process. For high-impact incidents, use your organization's forensic tooling to capture RAM before shutdown.

Windows loaded modules for a process

```
Get-Process -Id 1234 -Module | Select ModuleName,FileName
```

Linux open files and mappings

```
sudo lsof -p 1234
sed -n '1,240p' /proc/1234/maps
```

INJECTION CAVEAT: A legitimate signed executable can host injected or in-memory malicious code. A valid file signature therefore reduces one kind of suspicion but does not rule out process injection.

Chapter 33 - Lateral Movement and Neighboring Systems

Once privileged credentials or administrative channels are implicated, the incident may extend beyond the first host. Do not begin probing other systems aggressively. Use authorized logs and management systems to look for the same accounts, hashes, remote destinations, service names, scheduled-task names, SSH keys, and time-correlated logons.

Windows indicators to pivot on

- Account and logon source address
- Service name and binary path
- Scheduled task name and command
- SHA-256 hash
- Remote destination/domain
- PowerShell command fragments
- New local administrator names

Linux indicators to pivot on

- SSH key fingerprint
- Source IP and account
- systemd unit name/path
- Cron command
- Payload hash/path

- sudo activity
- Remote destination/domain

The objective is scope, not retaliation. Identify which systems show the same evidence and isolate or escalate them according to policy.

Chapter 34 - Worked Case: The Foreign-Country Browser Message

Symptom: a user opens a website and is told the service is unavailable in a particular country. After reconnecting a VPN, the site works. This is a useful example because it feels alarming but has a strong benign hypothesis.

8. Record the current public egress IP before changing the VPN.
9. Capture current socket-to-process mappings.
10. Record VPN server/exit information and DNS/proxy state.
11. Reconnect the VPN and record the new public IP.
12. Compare whether the website behavior changes at the same time as the egress IP.
13. If the symptom follows the public IP, prioritize VPN/geolocation investigation. If it does not, investigate DNS, proxy, IPv6, browser state, and process-level network behavior.
14. Only treat the event as C2 when independent host evidence supports unauthorized code/control.

Chapter 35 - Worked Case: Suspicious Outbound HTTPS

Symptom: an unknown external address appears on port 443. The disciplined response is not to block the address first. Map it to the owning process, inspect the process path and command line, verify its publisher/hash, identify the parent, and check persistence. If the process is a legitimate updater connecting to its vendor infrastructure, document and close the lead. If it is an unexplained binary in a writable directory with persistence and repeated beaconing, contain and expand the investigation.

Chapter 36 - Worked Case: Defender Finds a PUA

Symptom: Defender reports a potentially unwanted application in Downloads, backups, or the Recycle Bin. Check ThreatName, IsActive, DidThreatExecute, resource paths, and ActionSuccess. A non-executed, inactive PUA with successful remediation is a very different incident from a Trojan that executed and created persistence. Remove redundant copies from backups only after confirming retention requirements, scan the machine, and avoid turning a PUA alert into unsupported C2 attribution.

Part VI - Operator Checklists

Live Windows Checklist

☐ Record time, hostname, user, and symptom.

☐ Capture TCP/UDP sockets and routes.

☐ Map suspicious PIDs to path, command line, parent PID, and service.

☐ Hash and verify signatures of suspicious executables.

☐ Review Defender detections and status.

☐ Review Startup, Run keys, tasks, services, and WMI subscriptions.

☐ Review PowerShell, process creation, service install, and logon events.

☐ Capture recent suspicious file writes.

☐ Contain confirmed malicious activity without deleting evidence.

☐ Disable persistence and quarantine confirmed payloads.

☐ Rotate exposed credentials and patch entry point.

☐ Reboot, rescan, and validate no recurrence.

Live Linux Checklist

☐ Record time, hostname, user, and symptom.

☐ Capture ss/lsof network state, routes, and neighbors.

☐ Map suspicious PIDs to /proc executable, command line, cwd, parent tree, and user.

☐ Hash and identify suspicious files; verify package ownership when applicable.

☐ Review systemd units/timers, cron, shell profiles, autostart, and authorized_keys.

☐ Review journal, auth/secure logs, login history, and sudo activity.

☐ Inspect /tmp, /var/tmp, /dev/shm, and web roots for recent payloads.

☐ Correlate service logs with process start and file creation times.

☐ Contain network/account/service activity.

☐ Disable persistence, quarantine payloads, rotate credentials, and patch entry point.

☐ Reboot/restart as appropriate, rescan, and validate no recurrence.

Part VII - Syntax and Command Glossary

This glossary consolidates the commands used throughout the guide. Commands are intentionally defensive. Replace example PIDs, paths, service names, task names, users, and addresses with values from your own investigation.

Get-NetTCPConnection

Windows PowerShell. Lists TCP endpoints. Use -State Established for active connections or -State Listen for listeners.

```
Get-NetTCPConnection -State Established | Select
LocalAddress,LocalPort,RemoteAddress,RemotePort,OwningProcess
```

Get-NetUDPEndpoint

Windows PowerShell. Lists local UDP endpoints and owning PIDs.

```
Get-NetUDPEndpoint | Sort OwningProcess
```

Get-CimInstance Win32_Process

Windows PowerShell. Returns process metadata including PID, parent PID, executable path, and command line.

```
Get-CimInstance Win32_Process | Select ProcessId,ParentProcessId,Name,ExecutablePath,CommandLine
```

Get-AuthenticodeSignature

Windows PowerShell. Checks the Authenticode signature state of a file.

```
Get-AuthenticodeSignature C:\Path\program.exe
```

Get-FileHash

Windows PowerShell. Computes a cryptographic file hash. SHA-256 is the default incident-response choice.

```
Get-FileHash C:\Path\program.exe -Algorithm SHA256
```

Get-CimInstance Win32_StartupCommand

Windows PowerShell. Enumerates common startup command registrations.

```
Get-CimInstance Win32_StartupCommand | Format-List *
```

Get-ScheduledTask

Windows PowerShell. Enumerates scheduled tasks; inspect .Actions to see commands.

```
Get-ScheduledTask | Select TaskPath,TaskName,State
```

Disable-ScheduledTask

Windows PowerShell. Disables a scheduled task during containment.

```
Disable-ScheduledTask -TaskName SuspiciousTask -TaskPath \
```

Unregister-ScheduledTask

Windows PowerShell. Deletes a confirmed malicious scheduled task.

```
Unregister-ScheduledTask -TaskName SuspiciousTask -TaskPath \ -Confirm:$false
```

Get-CimInstance Win32_Service

Windows PowerShell. Lists services, account, start mode, process, and binary path.

```
Get-CimInstance Win32_Service | Select Name,State,StartMode,StartName,PathName
```

Stop-Service / Set-Service

Windows PowerShell. Stops and disables a confirmed malicious service.

```
Stop-Service BadService -Force; Set-Service BadService -StartupType Disabled
```

sc.exe delete

Windows. Deletes a confirmed malicious Windows service registration.

```
sc.exe delete BadService
```

Get-MpComputerStatus

Windows PowerShell. Shows Microsoft Defender state and signature/scan information.

```
Get-MpComputerStatus | Format-List *
```

Get-MpThreatDetection

Windows PowerShell. Shows Microsoft Defender detection history.

```
Get-MpThreatDetection | Sort InitialDetectionTime -Descending
```

Get-MpThreat

Windows PowerShell. Maps Defender threat IDs to names/status.

```
Get-MpThreat | Format-List *
```

Update-MpSignature

Windows PowerShell. Updates Defender signatures.

```
Update-MpSignature
```

Start-MpScan

Windows PowerShell. Starts Defender quick or full scan.

```
Start-MpScan -ScanType QuickScan
```

Get-WinEvent

Windows PowerShell. Queries Windows event logs using filters.

```
Get-WinEvent -FilterHashtable @{LogName='System'; Id=7045; StartTime=(Get-Date).AddDays(-7)}
```

wevtutil epl

Windows. Exports an event log to an EVTX file.

```
wevtutil epl Security C:\IR\Security.evtx
```

New-NetFirewallRule

Windows PowerShell. Creates a Windows Firewall rule. Use only for confirmed indicators.

```
New-NetFirewallRule -DisplayName "IR block" -Direction Outbound -Action Block -RemoteAddress
203.0.113.10
```

Get-LocalGroupMember

Windows PowerShell. Lists members of a local group.

```
Get-LocalGroupMember -Group Administrators
```

Get-LocalUser

Windows PowerShell. Lists local users and selected account metadata.

```
Get-LocalUser | Select Name,Enabled,LastLogon,PasswordLastSet
```

ss

Linux. Shows sockets. -t TCP, -u UDP, -p process, -n numeric, -a all.

```
sudo ss -tupna
```

lsof -i

Linux. Shows processes with network files/sockets.

```
sudo lsof -nP -i
```

ps

Linux. Shows process metadata and command lines.

```
ps -eo pid,ppid,user,lstart,cmd --forest
```

pstree

Linux. Displays parent-child process relationships.

```
pstree -aps 1234
```

/proc/<pid>/cmdline

Linux. Contains the raw process command line separated by NUL bytes.

```
tr '\0' ' ' < /proc/1234/cmdline; echo
```

/proc/<pid>/exe

Linux. Symlink to a process executable.

```
readlink -f /proc/1234/exe
```

sha256sum

Linux. Computes SHA-256 hash of a file.

```
sha256sum /path/to/suspect
```

file

Linux. Identifies file type using signatures/magic.

```
file /path/to/suspect
```

stat

Linux. Shows ownership, permissions, size, and timestamps.

```
stat /path/to/suspect
```

systemctl

Linux. Controls and inspects systemd units.

```
systemctl cat suspicious.service
```

journalctl

Linux. Queries systemd journal.

```
sudo journalctl --since "24 hours ago"
```

crontab

Linux. Displays or edits user cron jobs.

```
crontab -l
```

find

Linux. Searches filesystem by path, time, type, permissions, and more.

```
sudo find /tmp /var/tmp /dev/shm -type f -mtime -2 -ls
```

last / lastb

Linux. Shows successful and failed login history where databases are available.

```
last -ai | head -n 100
```

grep

Linux. Searches text/logs for patterns.

```
grep -iE "Accepted|Failed|sudo" /var/log/auth.log
```

dpkg -S / rpm -qf

Linux. Maps a file to the package that owns it.

```
dpkg -S /usr/bin/example
```

dpkg -V / rpm -V

Linux. Verifies installed files against package metadata.

```
rpm -V package-name
```

lsof +L1

Linux. Finds open files whose link count is below one, including deleted-but-running executables.

```
sudo lsof +L1
```

nft

Linux. Manages nftables firewall rules. Use cautiously on remote systems.

```
sudo nft list ruleset
```

usermod -L

Linux. Locks a local user password; does not revoke keys/tokens/sessions.

```
sudo usermod -L username
```

ssh-keygen -lf

Linux. Shows fingerprints for SSH public keys.

```
ssh-keygen -lf /home/user/.ssh/authorized_keys
```

curl

Windows/Linux. Makes HTTP(S) requests. In this guide, used to reveal public egress IP.

```
curl -s https://api.ipify.org
```

arp -a / ip neigh

Windows/Linux. Shows neighbor/ARP cache.

```
arp -a   |   ip neigh show
```

route print / ip route

Windows/Linux. Shows routing table.

```
route print   |   ip route show
```

Appendix A - Suspicion Scoring Without Panic

A simple scoring model can keep operators from overreacting to one weird detail. Add confidence when independent signals agree; subtract confidence when a normal explanation fits the evidence.

Observation	Typical weight
Foreign/geolocated IP only	Very low
Unsigned binary in user-writable path	Moderate
Unknown persistence launching same binary	High
Process tree consistent with RCE	High
Known-malicious indicator plus matching process behavior	High
Credential theft/EDR tampering evidence	Very high
Activity disappears only when legitimate VPN exit changes	Supports network/geolocation explanation, not C2 by itself

Appendix B - Incident Notes Template

Incident ID / hostname: _____

Operator: _____

Start time and timezone: _____

Trigger / symptom: _____

Public egress IP: _____

Suspicious remote endpoints: _____

Suspicious PIDs/processes: _____

Relevant file hashes: _____

Persistence mechanisms: _____

Accounts affected: _____

Containment actions: _____

Evidence exported: _____

Eradication actions: _____

Credentials rotated: _____

Patch/configuration changes: _____

Validation results: _____

Open questions: _____

Appendix C - GreyNOC Closing Doctrine

Good incident response is disciplined skepticism. Do not accept the first scary explanation, and do not accept the first reassuring explanation either. Build the story from independent evidence. The operator who can explain which process communicated, what launched it, how it persisted, what privileges it obtained, how it entered, and why the behavior stopped has a defensible conclusion. The operator who only has an IP country, a process name, or a hunch does not.

When you cannot establish trust after a privileged compromise, rebuilding from trusted media is often the most honest technical answer. When the evidence points to a VPN exit node, a legitimate updater, or a non-executed PUA, say that clearly too. GreyNOC practice is not about finding an attacker in every anomaly. It is about finding the truth quickly enough to act.